126 PRINTERS TESTED ▸ NEW LAB REPORT

MAY 1993

BYTE

THE WORLDWIDE COMPUTING AUTHORITY

BYTE Picks the Best Printer for:

- WORD PROCESSING
- DESKTOP PUBLISHING
- CAD
- PRESENTATIONS
- PORTABILITY
- SPREADSHEETS
- GRAPHICS
- WORKGROUPS

Pentium

Twice as fast as a 486. But does Intel offer the best engine for Windows NT?

EXCLUSIVE:

What You Should Know About LAN Security Page 118

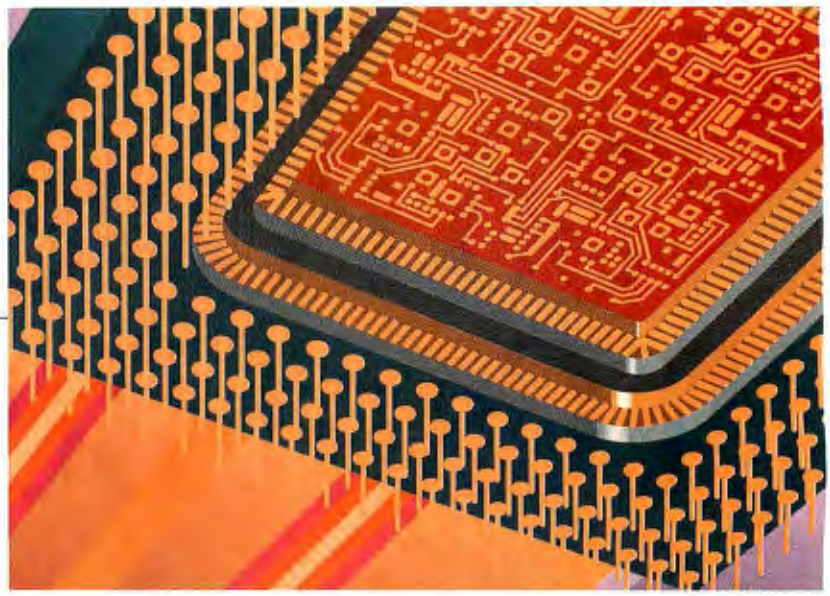
NEW:

- **Novell NetWare 4.0**
- **AutoCAD for Windows**

$3.50 U.S.A./$4.50 IN CANADA
A McGraw-Hill Publication/0360-5280

BYTE

May 1993 Volume 18, Number 6



COVER STORY

FEATURE

Intel Launches Rocket in a Socket

PAGE 92

NEWS

24 MICROBYTES
Bringing Unix into the NetWare fold presents Novell with a marketing dilemma: what to tell software developers who ask which platform to write applications for.

33 FIRST IMPRESSIONS
AutoCAD Draws on Windows
by Patrick Waurzyniak
Autodesk's best-selling CAD software goes Windows.

37 A Heavy-Hitting Unix for PCs
by Ben Smith
Solaris for Intel is the most complete PC Unix.

42 NetWare 4.0, the next step for a growing network

NoteJet 486, Canon's notebook/printer makes a dynamic package

HP LaserJet 4Si and 4Si MX, Hewlett-Packard's newest network printers

PagePlus 2.0, a $59.95 desktop publishing package

Compel, multimedia presentation software from Asymetrix

Painter 2.0, a "natural," professional, and fun paint tool

57 WHAT'S NEW
The DTR-1 is a notebook or a pen computer, the SmartLink V32bis FaxModem encrypts your data, LapCAD 5 for the Mac gives you finite modeling, and more.

FEATURES

92 Intel Launches Rocket in a Socket
by Tom R. Halfhill
Intel's new Pentium CPU doubles the speed of the fastest 486.

111 Oberon: A Glimpse at the Future
by Dick Pountain
A radically object-oriented design previews future operating systems.

STATE OF THE ART

SEEKING SECURITY

118 Overview: Seeking Security
by Paulina Borsook
Mainframe-type security is coming to the client/server environment.

129 Should Encryption Be Regulated?
by Peter Wayner
U.S. law enforcers want to limit your use of data encryption.

137 Stealth Virus Attacks
by John DeHaven
Anonymous attack software targets networked computers.

144 Resource Guide: Virus Protection for Networks

COVER IMAGE: PAUL AVIS © 1993

REVIEWS

146 **LAB REPORT: Hands-On Testing 126 PRINTERS**
by Richard Fox, Alan Joch, Chandrika Krishnamurthy, Stephen Platt, and Leonard Presberg
Our testing shows which printer is best for your needs.

176 **SOLUTIONS FOCUS**
Making the MPC Upgrade
by Greg Loveria
Some of the best options available for bringing multimedia to the PC.

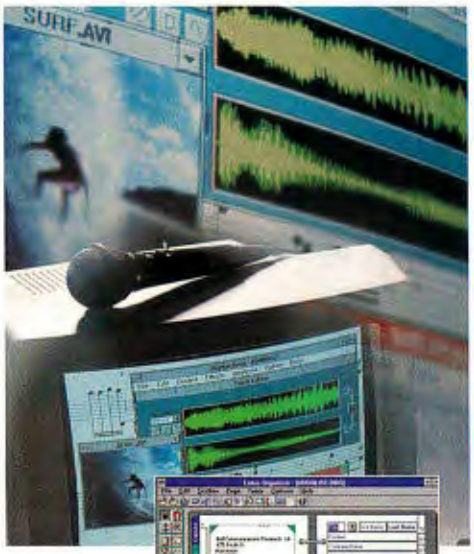
194 **Personable PIMs**
by Nicholas Baran
Ten PIMs for Windows, DOS, and the Mac, ranging from simple organizers to full-blown project managers.

199 **Banyan's "StreetTalk for NetWare"**
by John Rydberg
Banyan's distributed directory meets server-based NetWare.

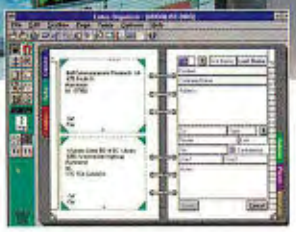
201 **Windows Video Recorders**
by Tom Yager
The hardware side of Video for Windows.

205 **Tape Backup on the Go**
by Howard Eglowstein
A review of four parallel-port tape drives for backup.

209 **Lotus 1-2-3 Release 3.4 for DOS**
by Nicholas John Delonas
Lotus' latest high-end DOS spreadsheet is faster, but you still may prefer the low-end line.



PAGE 176



PAGE 194

HANDS ON

213 **UNDER THE HOOD**
Computing on Wheels
by Dick Pountain
This month's column literally goes under the hood to examine a new generation of automotive-control computers.

221 **SOME ASSEMBLY REQUIRED**
The Power of Inheritance
by Allen Holub
How to take advantage of multiple inheritance in your C++ class designs.

229 **BEYOND DOS**
Getting a Handle on NT
by Gen Kiyooka,
Windows NT offers outstanding exception-handling tools. Here's why you need them.

OPINIONS

75 **USER'S COLUMN**
Once More into the Breach
by Jerry Pournelle
User's Choice Awards wrap-up.

210 **BOOK AND CD-ROM REVIEWS**
From the Mind of MIT
by Hugh Kenner, Steve Apiki, and Tom Yager
Windows NT, multimedia, MIT, Compton's on CD-ROM, and more.

282 **STOP BIT**
Losing the Human Edge
by Hans Berliner
Computers are about to mount a new challenge in the chess world.

12 **EDITORIAL**
Our New Lab Report: Your New Consultant
by Dennis Allen

20 **LETTERS**
Coming to terms with software gluttony; OS/2 hassles; machine translation; and other topics.

READER SERVICE

280 Editorial Index by Company
276 Alphabetical Index to Advertisers
278 Index to Advertisers by Product Category
Inquiry Reply Cards: **276A**

233 **BUYER'S GUIDE**
Mail Order
Hardware/Software Showcase
Buyer's Mart

PROGRAM LISTINGS

From BIX: Join "listings/frombyte93" and select the appropriate subarea (i.e., "may93"). From the UUNET: ftp to ftp.uu.net, log on as anonymous," and enter your user ID as your password. Type "cd/published/byte" and type "DIR." Files appear in subdirectories arranged by month.

From the BYTE BBS at 1200–9600 bps: Dial (603) 924-9820 and follow the instructions at the prompt.

INSIDE BYTE

PAGE 8

BYTE (ISSN 0360-5280) is published monthly with additional issues in April and October by McGraw-Hill, Inc. U.S. subscriber rate $29.95 per year. In Canada and Mexico, $34.95 per year. European surface mail subscriptions $60, airmail $80. Non-European subscriptions, $60 surface mail or $85 airmail. All foreign subscriptions are payable in U.S. funds that can be drawn on a U.S. bank. Single copies $3.50 in the U.S., $4.50 in Canada. Executive, Editorial, Circulation, and Advertising Offices: One Phoenix Mill Lane, Peterborough, NH 03458. Second-class postage paid at Peterborough, NH, and additional mailing offices. Postage paid at Winnipeg, Manitoba. Registration number 9321. Registered for GST as McGraw-Hill, Inc., GST #123075673. Printed in the United States of America. Postmaster: Send address changes and fulfillment questions to BYTE Subscriptions, P.O. Box 552, Hightstown, NJ 08520.

GEN KIYOOKA

GETTING A HANDLE ON NT

Businesses cannot afford software systems that make informal or implicit assumptions about the conditions in which they execute. By formally including such assumptions, you let the software behave robustly. The operating systems, languages, and tools simply afford varying degrees of convenience for expressing the subtleties of the design.

Windows NT's proposed C-language extensions and structured exception-handling model provide excellent design tools for meeting this goal. Win32 under NT introduces concisely expressed solutions to design problems that have impeded the development of vigorous software programs and plagued Windows 3.x with reliability problems. When used with a good design, these tools produce a software system of unparalleled reliability.

The Object Model

A good exception-handling model does nothing without a good software design. An object-oriented model provides the foundation for your software design because it lets you build reliability and robustness into a set of reusable software components. Because software consists of micro objects (e.g., a single integer variable) within macro objects (e.g., a DLL) within system objects (e.g., applications that interact on the desktop), the object model must apply design considerations that affect system reliability in general. Any component not properly designed should be prevented from having an adverse effect on other parts of the system.

An object is merely a packet of data around which you wrap the set of methods used to manipulate the state of the object's data. Any implementation that fails to observe the exclusivity of access to data via an object's methods circumvents the design assumptions and places the reliability of the software in question. For any object in a software system, it can be said that the private data can assume a finite number of states that fall into two categories: valid states and invalid states. An object's methods provide a reliable means of transforming the private data from one valid state to another, ensuring behavior consistent with the design.

A general definition of an exception in an object-oriented system might say that the private data of an arbitrary object has reached a state for which the assumptions of the designer do not hold. My experience suggests that bug reports are merely observations of a software program attempting to apply object methods to private data that has reached some invalid state. The behavior that the designer promised applies only to objects in valid states, and the effect of applying methods to invalid data is unpredictable, undesirable, and usually downright embarrassing.

Windows NT makes exception handling easier for applications developers

In his book *Object-Oriented Software Construction* (Prentice-Hall, 1988), Bertrand Meyer provides a compelling argument for explicitly including design assumptions in software. He offers a conceptual model of system health in the form of exception detection and the prevention of object transitions into invalid states. Because an object's methods exclusively control the state transitions of its data, the methods form an appropriate place to implement design assumptions.

A method invoked on an object can therefore result in one of two outcomes: successful or unsuccessful. The criteria for determining success come from the preconditions and postconditions of the method, assertions that must hold for an operation to be considered successful. If any precondition or postcondition does not hold, an exception has been detected.

The liberal use of preconditions precludes the need for postconditions and provides an effective debugging tool. A complete set of preconditions ensures that an object will never go into an invalid state and thus fail to carry the operation to completion. I use preprocessor macros to define a `PRECONDITION ( expression, returnValue )` to simplify the declaration of these explicit assumptions. I base my precondition macro on the standard `assert()` macro declared in the ANSI C `assert.h`.

However, there is a difference between a true precondition, which alters program flow in both debug and production builds, and the assertion macro, which takes a benign role in production code. Murphy's Law explains that any assertion that failed to appear during debugging is certain to occur once the product is released.

Given a robust model, the problem of reliability can be reduced to these four problems: exception detection, typing, cleanup, and handling. You can detect most program exceptions by a complete

ILLUSTRATION: DAVID CHEN © 1993

set of preconditions in the program code. In the absence of an intrinsic mechanism, exception conditions can be typed via return values and error codes. Here, the caller acts on the exception, most often by propagating an indication of the operation's failure back to the boundary of the system that received the event. Usually, an operation foiled by early detection is not a problem. The longer an invalid object state goes undetected and unreported, the greater the chance of data corruption.

Beyond the Model

In the absence of a general conceptual model (e.g., the one proposed by Meyer) and elegant operating-system services (e.g., the ones provided by Windows NT), exceptions are considered infrequent situations for which you use special function calls to save and restore a minimum execution state. Standard C libraries provide `setjmp` and `longjmp`, and Windows 3.x includes `Catch` and `Throw`. The execution state that these functions save does not include the state of objects that might be acted on by the miscreant code. Therefore, you must limit the use of these functions to prevent the exception handler's negative side effects from occurring undetected later in the program.

Floating-point libraries often provide facilities for hooking the exception handler. Under 16-bit Windows, you can handle memory-protection and divide-by-zero violations by using the ToolHelper DLL, a component designed to help programmers develop debugging tools. ToolHelper merely provides a means for identifying the exception after the fact. You must rely on `Catch` and `Throw` to give logical structure to the flow of execution.

This limitation is loosely enforced by the need for `setjmp` and `longjmp` to reference a common symbol representing the buffer where the execution state is stored. These services provide handling for exception conditions that can be readily anticipated and identified.

The absence of a robust operating-system design places a burden on applications software developers. Many choose to ignore this, designing software without explicitly expressing underlying assumptions. It would be a mistake for these developers not to apply the Win32 exception-handling services. NT's exception-handling services provide a consistent, reliable model for reporting and handling exceptions detected by the hardware, operating-system software, third-party DLLs, subsystems, and programming libraries in use by your software, and those subsystems your own code defines and detects.

Windows NT's structured exception-handling model provides a clean, reliable method of unwinding nested program execution at the advent of an unhandled exception. It supports an elegant method for implementing selective handlers and provides a guaranteed means for releasing system resources allocated during the course of execution that lead up to an exception condition. The NT model conveniently proposes a few simple extensions to the C-language syntax, along with some intrinsic functions that will require standardization to maintain source code portability to other operating systems.

Guarding Your Code

Under Win32 and the Microsoft C implementation of the syntax extensions, you can guard code segments against exception conditions with the `try-except` statement. The `try` keyword sets forth a compound statement block that is protected against exception conditions. This protection extends to nested functions called within the protected block of code. Immediately following the closing brace of the guarded code, the `except()` statement receives an argument capable of selectively filtering exceptions for the handler, the compound statement block occurring immediately after the `except()` statement.

The absence of a robust operating-system design places a burden on software developers.

After identifying an exception condition, the operating system searches backward through the frames of execution history, searching for a handler that's capable of dealing with the exception. Exceptions are typed by a single 32-bit word obtained from a special function, `GetExceptionCode()`, whose meaning is of value only inside the exception filter. An arbitrary amount of exception-specific information is also available by way of a call to `GetExceptionInformation()`.

This system allows for the development of rich, capable user-defined exceptions (invoked by the `RaiseException()` function) and complex filtering functions. Yet it maintains a concise declaration for simpler, more common scenarios. Given that exception handlers adeptly handle nested execution by implying that the execution be unwound back to the first capable handler, any critical, shared system resources allocated during the inward course leading up to an exception must be freed as the execution frames are unwound.

Again, this provision is expressed by a new language construct, called the `try-finally` statement. As with `try-except`, the `try` keyword precedes a block of code to be guarded against exceptions. The `finally` keyword sets forth a block of code that the operating system/compiler implementer has guaranteed will execute, regardless of whether an exception occurred or the whereabouts of the handler.

Applied Knowledge

There's no excuse for not including robust architecture by design. Relying on the types of exceptions that the operating system and processor hardware identify is hardly a starting point for healthy software. By the time an uninitialized pointer gets to the point of generating a memory-protection violation, it has probably already trashed several of your software's most crucial data structures. Even if the execution is unwound to the satisfaction of a programmer concerned with keeping the software alive, users are likely to go on and destroy their chances for data recovery.

Meyer's model gives a practical means for interrogating object instances to force early exception detection. The arrival of Windows NT makes handling common exceptions more convenient. But the real question is how well you can apply forward-thinking, object-oriented techniques to early detection of exceptions, using the consistency of `RaiseException()` to unify exception-handling procedures. Such an approach is likely to produce robust code at the expense of concise declaration and execution speed.

The art of identifying circumstances and judicious application of the different approaches is a matter of good design. It's a good feeling knowing that the best design is relatively easy to implement. The only time I get that feeling is when I'm developing software for Windows NT. ■

Gen Kiyooka is the founder of Electron Image and the inventor of Blue Sky Software's RoboHelp and BugMan programming tools. You can reach him on BIX c/o "editors" or on CompuServe at 76376,43.